

পুরোনো tokenizer থেকে নতুন tokenizer ট্রেন করা

যদি কোনো language model আপনার চাওয়া ভাষার জন্য না থাকে, বা আপনার data আগের model-এর training data থেকে অনেক আলাদা হয়, তাহলে আপনাকে নতুন করে tokenizer বানাতে হতে পারে।

সহজভাবে বললে:

- model নতুন করে train করতে হলে
- আগে tokenizer-ও নতুন করে train করতে হয়

কারণ tokenizer-ই ঠিক করে text-কে কীভাবে ছোট ছোট token বা subword-এ ভাঙা হবে।

tokenizer training আসলে কী?

Transformer model সাধারণত text-কে পুরো word হিসেবে নেয় না। বরং text-কে ছোট ছোট subword-এ ভাঙে।

যেমন tokenizer-এর কাজ হলো বের করা:

- কোন subword সবচেয়ে বেশি ব্যবহার হয়
- কোন subword data-র জন্য সবচেয়ে useful

এজন্য tokenizer-কে পুরো corpus-এর সব text দেখে শিখতে হয়।

এই শেখার process-কেই tokenizer training বলে।

খুব গুরুত্বপূর্ণ কথা

Tokenizer training আর model training এক জিনিস না।

Model training

Model training-এ:

- stochastic gradient descent ব্যবহার হয়
- loss কমানো হয়
- process কিছুটা random হয়
- তাই একই training দুইবার চালালে result একটু আলাদা হতে পারে
- একই result চাইলে seed set করতে হয়

Tokenizer training

Tokenizer training-এ:

- text analysis করা হয়
- কোন subword রাখা হবে সেটা ঠিক করা হয়
- এটা statistical process
- এটা deterministic

মানে:

একই algorithm + একই corpus দিলে সবসময় একই result আসবে।

Corpus বানানো

Transformers library-তে একটি সহজ API আছে:

```
AutoTokenizer.train_new_from_iterator()
```

এটা দিয়ে পুরোনো tokenizer-এর মতো বৈশিষ্ট্য রেখে নতুন tokenizer train করা যায়।

ধরি, আমরা GPT-2 নতুন করে train করতে চাই, কিন্তু সাধারণ English-এর বদলে অন্য ধরনের data দিয়ে।

এখানে example হিসেবে natural language না নিয়ে Python code ব্যবহার করা হয়েছে।

মানে এখানে language হিসেবে ধরা হয়েছে Python source code।

Dataset load করা

Python code-এর data জোগাড় করতে **datasets** library ব্যবহার করা হয়েছে।

CodeSearchNet dataset-এ GitHub-এর open-source project থেকে অনেক function আছে।

এখানে Python অংশ load করা হচ্ছে:

```
from datasets import load_dataset
```

```
raw_datasets = load_dataset("code_search_net", "python")
```

এটা load হতে একটু সময় লাগতে পারে।

Training split-এ কী আছে?

```
raw_datasets["train"]
```

Output হবে এরকম:

```
Dataset({
  features: ['repository_name', 'func_path_in_repository', 'func_name', 'whole_func_string',
'language',
  'func_code_string', 'func_code_tokens', 'func_documentation_string',
'func_documentation_tokens', 'split_name',
  'func_code_url'
],
  num_rows: 412178
})
```

এখানে বোঝা যায় dataset-এর মধ্যে অনেক column আছে।

আমরা tokenizer training-এর জন্য ব্যবহার করব:

`whole_func_string`

কারণ এতে পুরো function একসাথে আছে।

একটি **example** দেখা

```
print(raw_datasets["train"][123456]["whole_func_string"])
```

এতে একটি Python function print হবে।

এখানে idea হলো:

আমরা tokenizer-কে Python code শেখাবো।

এখন মূল কাজ

আমাদের dataset-কে এমনভাবে tokenizer-এর কাছে দিতে হবে, যাতে:

- একবারে একটু একটু data যায়
- সব data একসাথে RAM-এ না ওঠে
- memory শেষ না হয়ে যায়

এই জন্য iterator বা generator ব্যবহার করা হয়।

কেন **list of texts** দরকার?

Tokenizer যদি একবারে batch আকারে text পায়, তাহলে faster কাজ করে।

তাই আমরা text-গুলো একেক batch-এ দেব।

যেমন:

প্রতি batch-এ 1000টা text।

যেটা করা ঠিক না

নিচের code সব data memory-তে load করে ফেলবে:

```
# training_corpus = [raw_datasets["train"][i: i + 1000]["whole_func_string"] for i in range(0, len(raw_datasets["train"]), 1000)]
```

এটা ছোট dataset হলে ঠিক আছে।

কিন্তু বড় dataset হলে RAM problem হতে পারে।

1) এই লাইনের কাজ এক কথায়

এই লাইনটা `raw_datasets["train"]` ডেটাসেটকে 1000 করে ছোট ছোট ভাগে কাটছে, তারপর প্রতিটা ভাগ থেকে `"whole_func_string"` নামের কলামের ডাটা বের করে `training_corpus` নামে একটি লিস্ট রাখছে।

মানে:

- পুরো train dataset নেওয়া হচ্ছে
- 1000 সাইজের chunk বানানো হচ্ছে
- প্রতিটা chunk থেকে শুধু `"whole_func_string"` field নেওয়া হচ্ছে
- সব chunk মিলিয়ে একটা list তৈরি হচ্ছে

2) একদম ছোট ছোট অংশে ভাগ করলে

এখানে তিনটা বড় অংশ আছে:

A. `range(0, len(raw_datasets["train"]), 1000)`

এটা index তৈরি করছে।

ধরি train dataset এ 4500টা row আছে।

তাহলে এটা দেবে:

0, 1000, 2000, 3000, 4000

মানে:

- প্রথম chunk শুরু হবে index 0 থেকে
- দ্বিতীয় chunk শুরু হবে 1000 থেকে
- তৃতীয় chunk শুরু হবে 2000 থেকে
- এভাবে চলবে

এখানে:

- 0 = শুরু
- `len(raw_datasets["train"])` = dataset এর শেষ পর্যন্ত
- 1000 = এক ধাপ করে 1000 করে বাড়বে

B. `raw_datasets["train"][i: i + 1000]`

এটা dataset থেকে 1000 rows এর slice নিচ্ছে।

যদি `i = 0` হয়:

```
raw_datasets["train"][0:1000]
```

মানে প্রথম 1000টা row

যদি `i = 1000` হয়:

```
raw_datasets["train"][1000:2000]
```

মানে পরের 1000টা row

যদি `i = 4000` হয়:

```
raw_datasets["train"][4000:5000]
```

কিন্তু যদি dataset এ 4500 row-ই থাকে, তাহলে এটা 4000 থেকে 4499 পর্যন্ত যত আছে ততই নেবে।

C. `["whole_func_string"]`

এটা slice করা chunk থেকে "whole_func_string" নামের column বের করছে।

মানে, ওই chunk এর সব row থেকে শুধু এই field-এর data নেওয়া হচ্ছে।

ভালো উপায়: **generator** ব্যবহার করা

```
training_corpus = (  
    raw_datasets["train"][i : i + 1000]["whole_func_string"]  
    for i in range(0, len(raw_datasets["train"]), 1000)  
)
```

এখানে কী হচ্ছে?

- সাথে সাথে data load হচ্ছে না
- শুধু একটি generator object তৈরি হচ্ছে
- যখন দরকার হবে, তখন 1000টি text load হবে
- আবার পরের দরকারে পরের 1000টি load হবে

এভাবে memory বাঁচে।

Generator-এর limitation

Generator একবারই ব্যবহার করা যায়।

দেখো:

```
gen = (i for i in range(10))  
print(list(gen))  
print(list(gen))
```

Output:

```
[0, 1, 2, 3, 4, 5, 6, 7, 8, 9]  
[]
```

প্রথমবার data পাওয়া গেল।

দ্বিতীয়বার কিছুই পাওয়া গেল না।

কারণ generator একবার exhaust হয়ে গেলে শেষ।

তাই **function** বানানো হয়

এই problem solve করার জন্য generator-কে function-এর ভিতরে রাখা হয়।

```
def get_training_corpus():  
    return (  
        raw_datasets["train"][i : i + 1000]["whole_func_string"]  
        for i in range(0, len(raw_datasets["train"]), 1000)  
    )
```

```
training_corpus = get_training_corpus()
```

এখন যখনই `get_training_corpus()` call করবে, নতুন generator পাবে।

আরেকটা **clean way: yield** ব্যবহার করা

```
def get_training_corpus():  
    dataset = raw_datasets["train"]  
    for start_idx in range(0, len(dataset), 1000):  
        samples = dataset[start_idx : start_idx + 1000]  
        yield samples["whole_func_string"]
```

এটাও একই কাজ করে।

এখানে:

- `dataset` মানে training data
- `start_idx` 0, 1000, 2000 এভাবে বাড়ে
- প্রতি step-এ 1000টা sample নেওয়া হয়
- তারপর `whole_func_string` return করা হয়
- `yield` থাকার কারণে এটা generator হিসেবে কাজ করে

নতুন **tokenizer** ট্রেন করা

- $\dot{G}$ মানে space

- `\n` মানে newline

এখানে **problem** কোথায়?

এই tokenizer code-এর জন্য খুব efficient না।

কারণ:

- প্রতিটি space আলাদা token বানাচ্ছে
- indentation একসাথে না নিয়ে ভেঙে ফেলছে
- code-এ 4 বা 8 space অনেক common, তাই এগুলো একসাথে token হলে ভালো হতো
- function name `add_numbers`-কে একটু অদ্ভুতভাবে split করেছে
- `_` character-এর সাথে tokenizer খুব familiar না

মানে plain English-এর জন্য বানানো tokenizer code-কে naturalভাবে handle করতে পারছে না।

নতুন **tokenizer train** করা

এখন আমরা নতুন tokenizer train করব।

```
tokenizer = old_tokenizer.train_new_from_iterator(training_corpus, 52000)
```

এখানে:

- `training_corpus` হলো আমাদের data
- `52000` হলো নতুন vocabulary size

মানে tokenizer training শেষে প্রায় 52,000 token-এর vocabulary বানাবে।

এই **training slow** নাকি **fast**?

Corpus যদি বড় হয়, তাহলে কিছুটা সময় লাগবে।

কিন্তু fast tokenizer হলে এই কাজ surprisingly দ্রুত হয়।

খুব **important note**

`AutoTokenizer.train_new_from_iterator()` শুধু তখনই কাজ করবে, যখন tokenizer টি **fast tokenizer** হবে।

Fast tokenizer মানে কী?

Transformers library-তে দুই ধরনের tokenizer আছে:

1. Pure Python tokenizer

এগুলো পুরো Python-এ লেখা।

2. Fast tokenizer

এগুলো **Tokenizers** library-এর ওপর based
আর এই library Rust language-এ লেখা।

কেন fast tokenizer দরকার?

Python data science-এর জন্য খুব popular।

কিন্তু খুব fast parallel কাজ করার জন্য Python যথেষ্ট না।

যেখানে speed খুব গুরুত্বপূর্ণ, সেখানে অন্য language ব্যবহার করা হয়।

যেমন:

- model-এর matrix multiplication CUDA/C library দিয়ে হয়
- tokenizer training fast করতে Rust ব্যবহার করা হয়

Pure Python দিয়ে tokenizer train করলে কী হতো?

একদম slow হতো।

খুবই slow।

এই কারণেই **Tokenizers** library তৈরি করা হয়েছে।

Rust জানতেই হবে?

না।

যেমন GPU-তে model চালাতে CUDA language শেখা লাগে না, তেমনি fast tokenizer ব্যবহার করতে Rust শেখা লাগে না।

কারণ library-এর Python binding আছে।

মানে আপনি Python দিয়েই কাজ করবেন, ভিতরে ভিতরে Rust fast কাজ করে দেবে।

Fast tokenizer কী কী কাজে speed দেয়?

- নতুন tokenizer training
- batch tokenization
- parallel processing

AutoTokenizer কী করে?

AutoTokenizer সাধারণত fast tokenizer available থাকলে সেটাই automatically choose করে।

মানে বেশিরভাগ সময় আপনাকে manually ভাবতে হয় না।

নতুন tokenizer দিয়ে আগের example test করা

এখন নতুন tokenizer দিয়ে একই example tokenize করি:

```
tokens = tokenizer.tokenize(example)
tokens
```

Output:

```
['def', 'Ġadd', 'Ġ_', 'numbers', 'Ġ(', 'Ġa', 'Ġ,', 'ĠĠb', 'Ġ):', 'ĠĠĠĠĠ', 'ĠĠ"', 'Add', 'Ġthe', 'Ġtwo', 'Ġnumbers', 'ĠĠ', 'a', 'Ġ,', 'Ġand', 'ĠĠ', 'Ġb', 'Ġ.', 'Ġ"', 'ĠĠĠĠĠ', 'Ġreturn', 'Ġa', 'Ġ+', 'Ġb']
```

এখানে কী improvement হলো?

আবারও **Ġ** আর **Ġ** আছে।

কিন্তু এবার tokenizer Python code-এর কিছু special pattern শিখে ফেলেছে।

যেমন:

- **ĠĠĠĠĠ** = newline + indentation
- **Ġ" " "** = docstring শুরু হওয়ার triple quote

এগুলো Python code-এ খুব common।

তাই tokenizer এখন এগুলোকে smart way-তে ধরতে পারছে।

function name handling-ও ভালো হয়েছে

আগে `add_numbers`-কে awkward way-তে split করছিল।

এখন tokenizer function name-টা ভালোভাবে handle করছে।

মানে code-specific structure শিখে ফেলেছে।

নতুন tokenizer বেশি compact

একই code tokenize করতে নতুন tokenizer কম token ব্যবহার করছে।

```
print(len(tokens))  
print(len(old_tokenizer.tokenize(example)))
```

Output:

27

36

এখানে দেখা যাচ্ছে:

- নতুন tokenizer: 27 token
- পুরোনো tokenizer: 36 token

মানে নতুন tokenizer same text-কে কম token-এ প্রকাশ করতে পারছে।

এটাই ভালো tokenizer-এর লক্ষণ।

কারণ:

- representation compact হয়
- unnecessary split কমে
- model code ভালোভাবে বুঝতে পারে

আরেকটা example

```
example = """class LinearLayer():  
    def __init__(self, input_size, output_size):  
        self.weight = torch.randn(input_size, output_size)  
        self.bias = torch.zeros(output_size)  
  
    def __call__(self, x):
```

```
        return x @ self.weights + self.bias
    """
tokenizer.tokenize(example)
```

Output:

```
['class', 'GLinear', 'Layer', '():', 'CGGG', 'Gdef', 'G_', 'init', '__(', 'self', ',', 'Ginput', '_', 'size', ',', 'Goutput', '_', 'size', '):', 'CGGGGGGG', 'Gself', ',', 'weight', 'G=', 'Gtorch', ',', 'randn', '(', 'input', '_', 'size', ',', 'Goutput', '_', 'size', ')', 'CGGGGGGG', 'Gself', ',', 'bias', 'G=', 'Gtorch', ',', 'zeros', '(', 'output', '_', 'size', ')', 'CCGGG', 'Gdef', 'G_', 'call', '__(', 'self', ',', 'Gx', '):', 'CGGGGGGG', 'Greturn', 'Gx', 'G@', 'Gself', ',', 'weights', 'G+', 'Gself', ',', 'bias', 'CGGGG']
```

এই **example**-এ কী বোঝা গেল?

এখানে tokenizer আরও ভালোভাবে Python structure ধরেছে।

1. indentation token শিখেছে

আগেই দেখেছি indentation-এর জন্য token আছে।

2. double indentation-ও শিখেছে

এখানে CCGGGGGGG এর মতো token আছে।

মানে tokenizer বুঝে গেছে:

newline-এর পরে অনেক space একসাথে আসা code-এ খুব common।

3. Python-এর common word এক token-এ ধরছে

যেমন:

- class
- init
- call
- self
- return

এগুলোকে আলাদা আলাদা meaningful token হিসেবে ধরছে।

4. _ এবং . দিয়েও ভালো split করছে

মানে snake_case আর object access দুটোই handle করতে পারছে।

যেমন:

- `input_size`
- `self.weight`

5. camel case-ও split করতে পারছে

যেমন:

`LinearLayer`

এটা tokenize হয়েছে:

`["GLinear", "Layer"]`

মানে tokenizer camel case-ও বুঝতে পারছে।

এটা অনেক useful।

tokenizer save করা

পরে আবার ব্যবহার করতে চাইলে নতুন tokenizer-টা save করে রাখতে হবে।

Model save করার মতোই tokenizer save করাও `save_pretrained()` method দিয়ে করা হয়।

```
tokenizer.save_pretrained("code-search-net-tokenizer")
```

এতে `code-search-net-tokenizer` নামে একটি নতুন folder তৈরি হবে।

এই folder-এর ভিতরে tokenizer আবার load করার জন্য যা যা file দরকার, সব থাকবে।

সহজভাবে বললে:

- tokenizer save হবে
- একটি folder তৈরি হবে

- পরে ওই folder থেকে tokenizer আবার load করা যাবে

অন্যদের সাথে **share** করতে চাইলে

আপনি চাইলে এই tokenizer Hugging Face Hub-এ upload করতে পারেন।

তাহলে আপনার colleague, friend, বা অন্য কেউও এটা use করতে পারবে।

এজন্য আগে Hugging Face account-এ login করতে হবে।

notebook-এর মধ্যে login করা

যদি notebook-এ কাজ করেন, তাহলে এই code ব্যবহার করতে পারেন:

```
from huggingface_hub import notebook_login

notebook_login()
```

এতে একটি widget দেখাবে, যেখানে আপনি Hugging Face login info দিতে পারবেন।

notebook-এ না হলে কী করবেন

যদি notebook-এ না কাজ করেন, তাহলে terminal-এ এই command লিখবেন:

```
huggingface-cli login
```

এতে terminal থেকে login করা যাবে।

login-এর পরে Hub-এ upload করা

login হয়ে গেলে tokenizer-টা Hub-এ push করতে পারবেন এই command দিয়ে:

```
tokenizer.push_to_hub("code-search-net-tokenizer")
```

এতে আপনার namespace-এর ভিতরে **code-search-net-tokenizer** নামে একটি নতুন repository তৈরি হবে।

সেখানে tokenizer-এর fileগুলো upload হয়ে যাবে।

পরে যেকোনো জায়গা থেকে **load** করা

Hub-এ upload হয়ে গেলে, পরে যেকোনো জায়গা থেকে `from_pretrained()` দিয়ে tokenizer load করা যাবে।

```
# নিচের "huggingface-course" এর জায়গায় আপনার নিজের namespace ব্যবহার করবেন
tokenizer = AutoTokenizer.from_pretrained("huggingface-course/code-search-net-tokenizer")
```

মানে:

- tokenizer local folder থেকে load করা যায়
- আবার Hugging Face Hub থেকেও load করা যায়

এই অংশের মূল কথা

নতুন tokenizer বানানোর পর:

1. `save_pretrained()` দিয়ে local এ save করতে হয়
2. চাইলে Hub-এ upload করা যায়
3. upload করার আগে login করতে হয়
4. পরে `from_pretrained()` দিয়ে সহজে আবার load করা যায়